

Submission Date: March 15, 2026

Project Milestone #3
4+1 View Model – Web API
SFWRTECH 4SA3 – Software Architecture

Submitted By:

Student Name: Amisha

Student ID: 400612546

Submitted To:

Professor Name: Kevin Browne

Learning Objectives

- Design and document a software architecture using the 4+1 view model.
- Implement a software architecture, including design patterns.
- Present a software architecture design.

Spendly – Smart Expense Tracker

Project Purpose:

Spendly is a smart yet simple expense tracking software that will be designed to help users record, organize, and review their personal spending. The purpose of the software is to allow users to log daily expenses, categorize them, keep track of spending over time, and view summarized reports such as monthly or category-based spending totals, including simple expected income and expense summaries based on recurring transactions. Spendly will focus on providing a minimal and easy-to-use system that supports basic accounting needs without unnecessary complexity, allowing non-business users to better understand their spending habits and manage their finances more effectively.

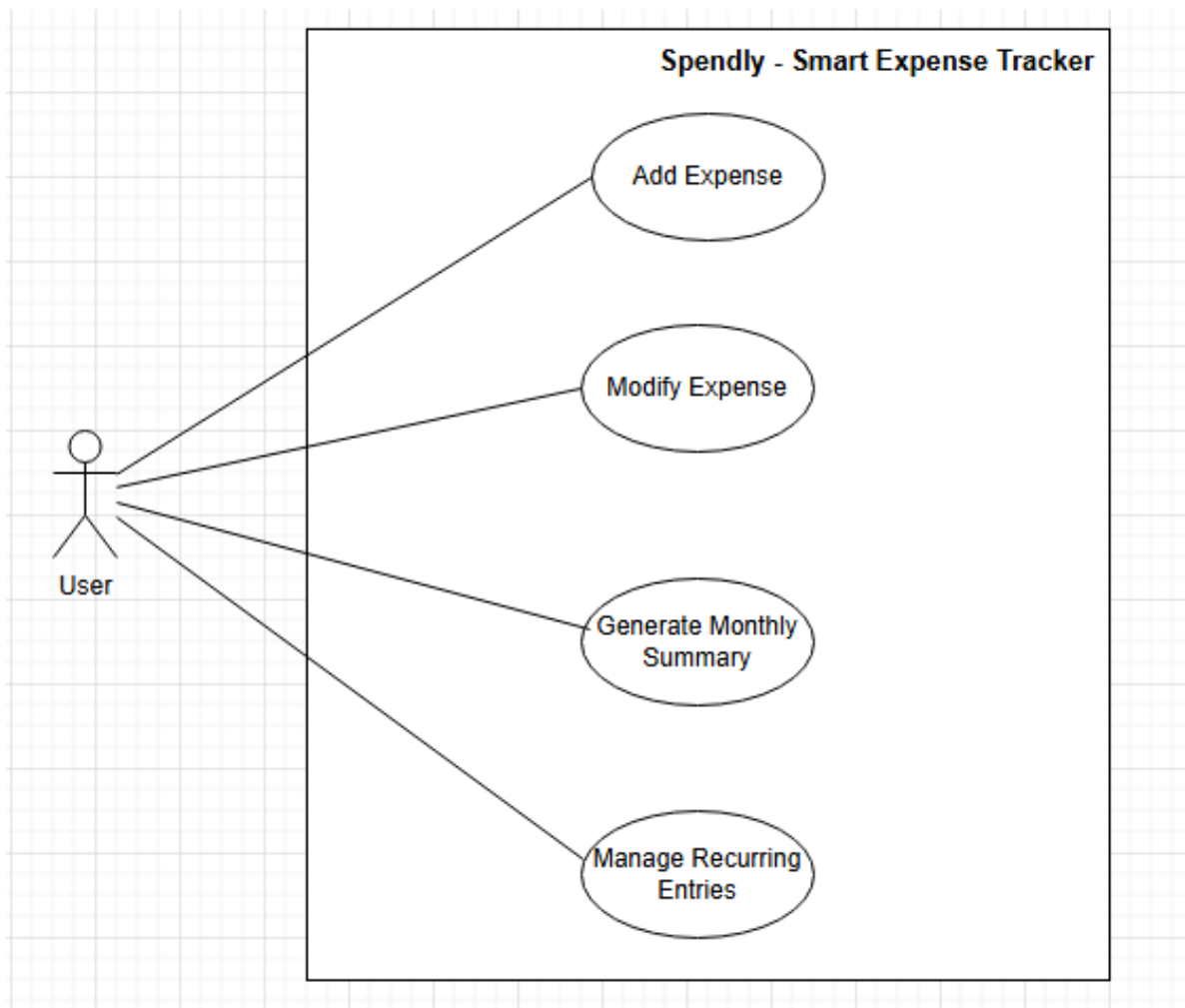
Target Audience:

The target audience for Spendly is non-accounting users such as students, working professionals, and individuals who manage personal finances on a regular income. The software is intended for people who want a simple way to track their expenses, understand where their money is going, and stay aware of upcoming income and recurring costs without needing accounting knowledge or complex financial tools.

Requirements:

ID	Name	Type	Description
R1	Add Expense	Functional	The system allows a user to add a new expense record including amount, date, and category.
R2	Modify Expense	Functional	The system allows a user to update or delete an existing expense record.
R3	Generate Monthly Summary	Functional	The system generates a monthly summary of total recorded expenses.
R4	Manage Recurring Entries	Functional	The system allows users to define recurring income and recurring expense entries.
R5	Cloud Data Storage	Non-Functional	The system stores and retrieves data using a cloud-based database service.
R6	Architectural Separation	Non-Functional	The system architecture maintains separation between user interface, business logic, and data storage components.

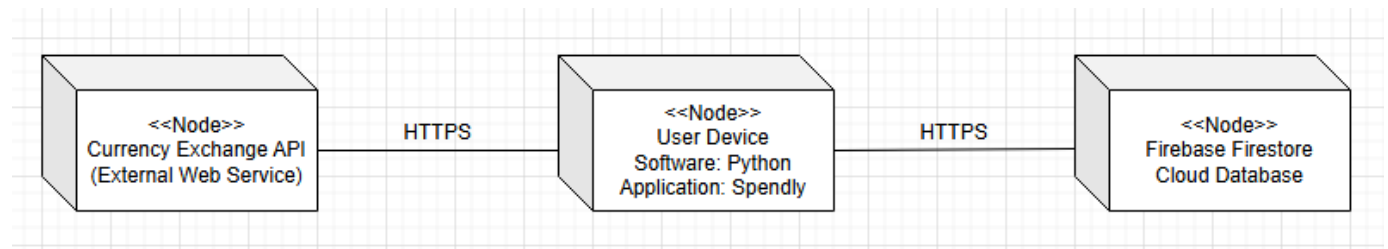
Scenario Viewpoint - UML Case Diagram:



Explanation: This Scenario Viewpoint above shows the way a user interacts with the Spendly system at a high level. In this diagram, the primary actor is the User using the application and they interact directly with the system to perform key financial tracking tasks. They can add expenses, modify existing expense records, generate monthly summaries, and manage recurring income and expense entries.

This viewpoint focuses on the system's functional requirements from the user's perspective. It highlights the main services that the system provides without showing internal implementation details or database interactions. The purpose of this view is to demonstrate how the system supports user goals through clearly defined use cases.

Physical Viewpoint - Deployment Diagram:



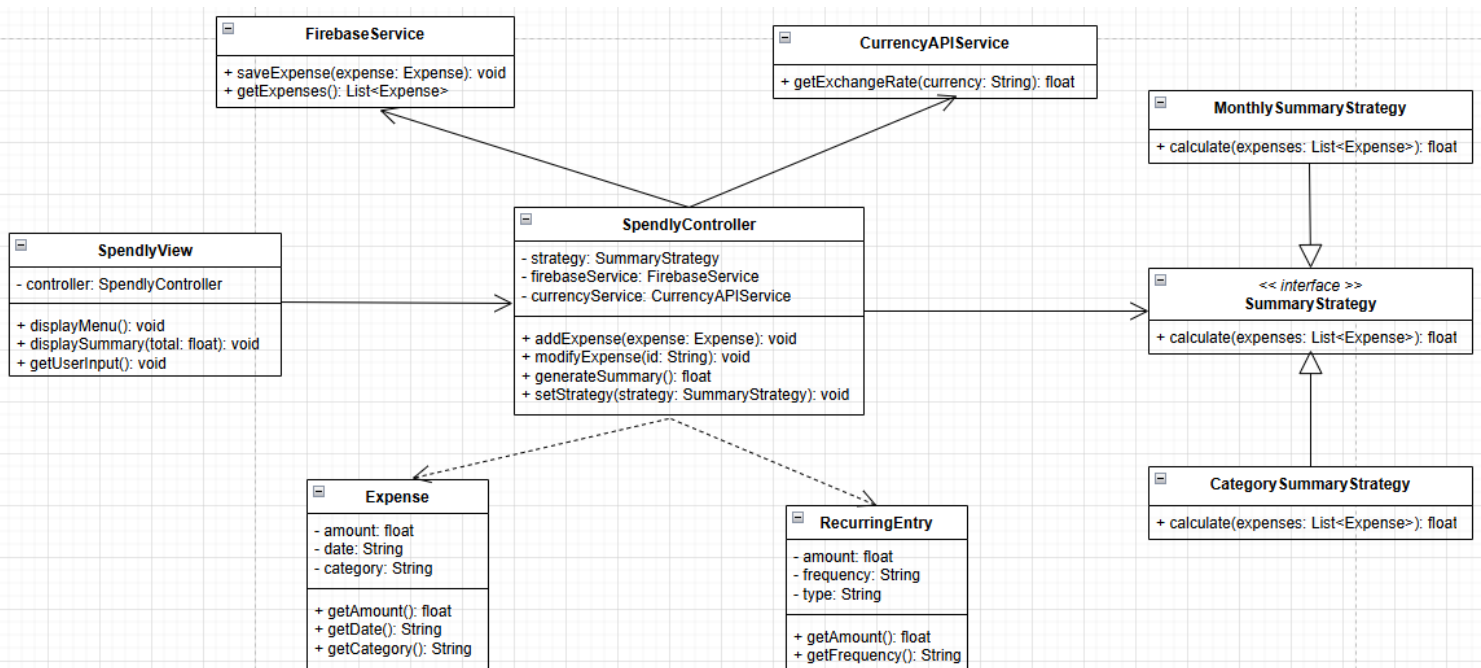
Explanation: This Physical Viewpoint above represents the deployment structure of Spendly system. The application runs on a user device as a Python-based program. This device hosts the Spendly application, which contains the core business logic and user interaction components.

It then communicates with Firebase Firestore, which serves as the cloud-based database responsible for storing and retrieving expense, income, and recurring transaction data. This communication occurs over HTTPS via the internet.

In addition, the application connects to a third-party currency exchange rate API using HTTPS. This external service provides up-to-date exchange rate information when currency conversion is required.

This viewpoint focuses on how system components are distributed across physical environments and demonstrates the interaction between the local application, cloud database, and external web services.

Development Viewpoint – UML Class Diagram:



Explanation: This Development Viewpoint above represents the internal code structure of the Spendly system from a programmer’s perspective. It illustrates how the main components of the system are organized and how responsibilities are separated within the application.

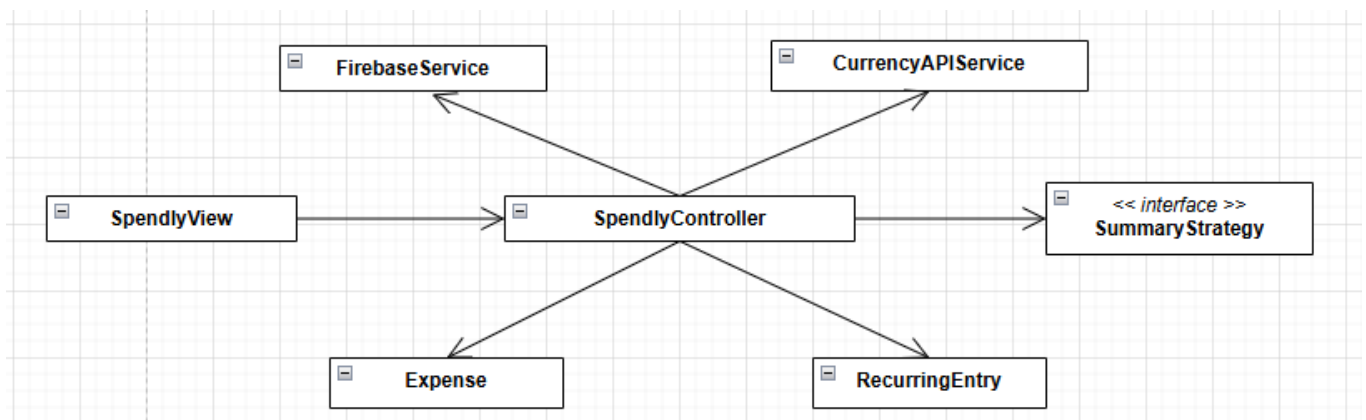
The architecture follows the Model–View–Controller (MVC) design pattern. The SpendlyView class handles user interaction and displays results. The SpendlyController manages system operations such as adding expenses, modifying expenses, and generating summaries. The Expense and RecurringEntry classes represent the core data models of the system and store expense-related information.

The system also applies the Strategy design pattern for calculating summaries. The SummaryStrategy interface defines a common method for summary calculation, while MonthlySummaryStrategy and CategorySummaryStrategy provide different implementations. This allows new summary calculation methods to be added without changing the controller logic.

Additionally, the FirebaseService class manages communication with the Firestore cloud database, and the CurrencyAPIService class handles interaction with the external currency exchange API. By separating these responsibilities into different classes, the architecture maintains clear separation between user interface, business logic, and data storage components.

This viewpoint focuses on how the software is structured at the code level and demonstrates how the system supports modularity, maintainability, and architectural separation.

Logical Viewpoint – UML Class Diagram:



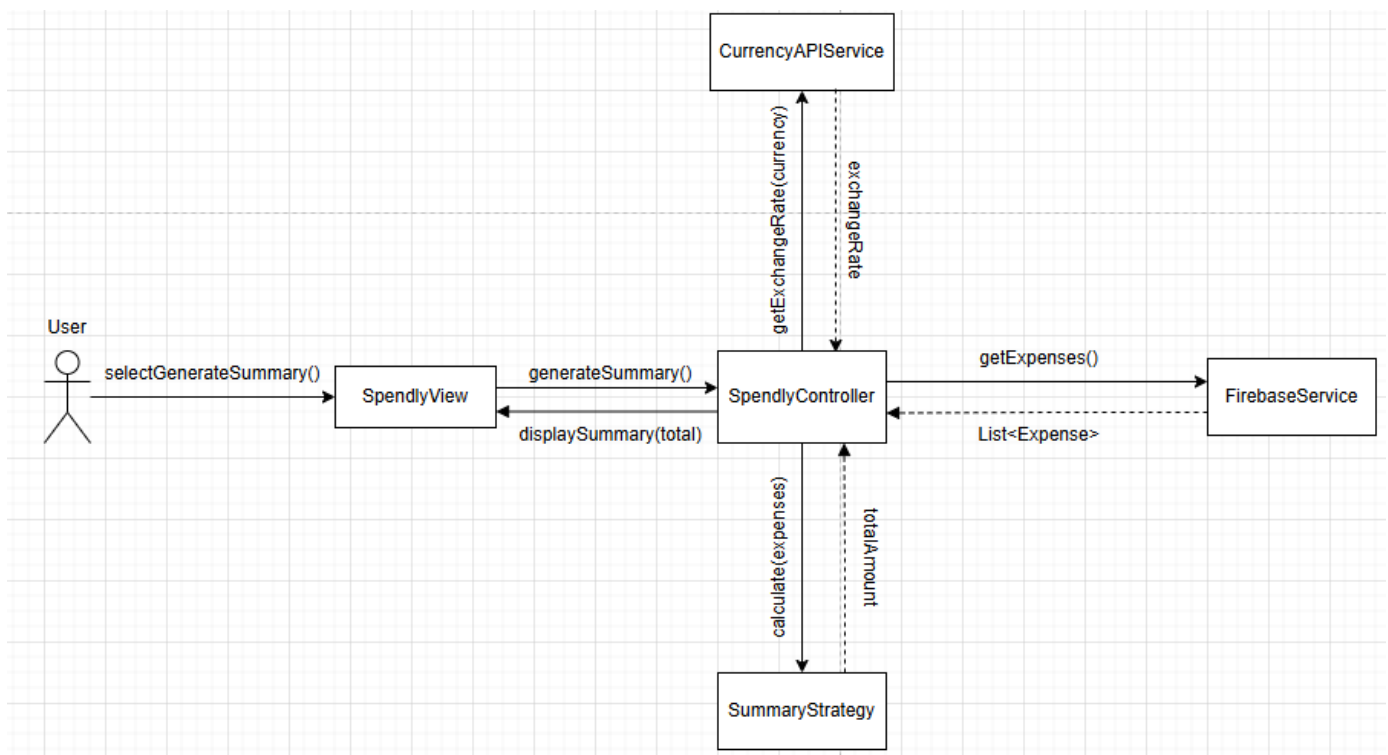
Explanation: This Logical Viewpoint above represents the conceptual structure of the Spendly system and the main responsibilities of its core components. It focuses on how the system is logically organized rather than how it is physically deployed or implemented in code.

The SpendlyView interacts with the SpendlyController, which coordinates system operations. The controller manages core domain entities such as Expense and RecurringEntry. It also communicates with FirebaseService for data storage and retrieval and interacts with CurrencyAPIService to obtain external currency exchange data when required.

The SummaryStrategy component represents the abstraction used for calculating financial summaries. The controller relies on this component to perform summary-related operations without being tightly coupled to a specific calculation method.

This viewpoint illustrates the main system components and their logical relationships, demonstrating how responsibilities are separated within the application.

Process Viewpoint - Sequence Diagram:



Explanation: This Process Viewpoint above illustrates the runtime interaction between components of the Spendly system when a user generates a monthly summary. It focuses on how responsibilities are distributed among the View, Controller, database service, strategy component, and external API.

The interaction begins when the User selects "Generate Summary" option in the SpendlyView. The view forwards this request to the SpendlyController using the `generateSummary()` operation.

The SpendlyController then coordinates the workflow. First, it requests stored expense data from the FirebaseService using `getExpenses()`. The FirebaseService returns a list of expense objects to the controller.

Next, the controller delegates the summary calculation to the SummaryStrategy component by calling `calculate(expenses)`. The strategy computes the total amount and returns to the controller.

After calculating the total, the controller requests the current currency exchange rate from the CurrencyAPIService using `getExchangeRate(currency)`. The external API service returns the exchange rate value.

Finally, once all required data has been processed, the SpendlyController sends the computed total to the SpendlyView using `displaySummary(total)`, and the result is presented to the user.

This viewpoint focuses on the dynamic behavior of the system and demonstrates how the controller orchestrates interactions between internal components and external services while maintaining separation of concerns.

Web API Demonstration Code:

To demonstrate integration with an external web service, a simple Python script was created to connect to a public currency exchange rate API. The script sends an HTTP request to the public API endpoint and retrieves real-time exchange rate data in JSON format.

The script then extracts the exchange rate value (USD to CAD) from the response and prints it to the console, formatted to two decimal places for better reading experience. Basic error handling is included to manage unsuccessful connections or invalid responses.

The purpose of this code is to verify that the Spendly application can successfully communicate with an external web API over HTTPS and process the returned data for use within the system.

❖ *Source code is added as .zip file in Avenue to Learn*